



天数智芯
Iluvatar CoreX

天数智芯

TorchDebug 精度调试工具使用指南

版本：V4.3.0 (x86)|V4.3.6 (ARM)

日期：2025.6.30

适用产品：天垓 150| 智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 TorchDebug 精度调试工具使用指南	4
1.1 修订记录	4
1.2 工具概述	4
1.3 工具使用流程	4
1.3.1 环境 1: 相同模型, 不同设备, 结果不同, 设备 0 结果正确, 设备 1 结果错误	5
1.3.2 环境 2: 相同模型, 同一设备, 不同软件栈版本的结果不同, 旧版本结果正确, 新版本结果错误	5
1.4 工具使用注意事项	5
1.5 工具使用示例	6
1.5.1 插入代码示例说明	6
1.5.2 手动模拟问题定位过程	8
1.6 相关接口说明	9
1.6.1 set_seed	9
1.6.2 init	10
1.6.3 start	11
1.6.4 stop	11
1.6.5 compare	11
1.6.6 analyze_op_diff_details	14
1.7 框架算子支持	14
1.8 最佳实践	15

1 TorchDebug 精度调试工具使用指南

1.1 修订记录

- COREX01-BI430-UG14-00: 2025/6/30
 - 更新**工具使用注意事项**
 - 新增**手动模拟问题定位过程** 使用示例
 - 更新**compare** API 接口的使用方法和参数说明
 - * 新增 compare_bounds 和 save_diff_details 参数
 - * 新增 Round_count 和 Diff_count 对比项
 - 删除 comparison_single_op API 接口, 用户可以使用**compare** API 接口中的 compare_bounds 实现同样效果
 - 新增**analyze_op_diff_details** API 接口
 - 新增 dump flash_attn **框架算子支持**
 - 新增**最佳实践**, 指导用户如何在进行大模型训练时使用 TorchDebug 进行精度调试
- COREX01-BI420-UG14-00: 2025/1/24
无内容更新
- COREX01-BI411-UG14-00: 2024/8/26
 - 新增"comparison_single_op"API 接口
- COREX01-BI410-UG14-00: 2024/6/30
文档首次对外发布

1.2 工具概述

天数智算软件栈提供 TorchDebug v0.1.0 调试工具, 该工具基于 PyTorch 并用于调试模型算子精度问题。开发者可以使用 TorchDebug 工具在整个网络中提取模型前向/反向计算过程中每个算子的输入与输出信息, 用于和参考对象的值进行比较, 并对导出的精度异常数据和精度正常数据中包含的 Tensor 信息进行比对和误差计算, 以定位计算有问题的算子。

TorchDebug 工具主要应用于以下场景:

- 因算子精度误差较大导致的确定性训练的网络精度问题
- 因轻微误差但经过后续算子的计算被放大导致的网络精度问题

1.3 工具使用流程

TorchDebug 工具可以应用于以下两种用户环境, 且两种用户环境的使用流程不同。

1.3.1 环境 1：相同模型，不同设备，结果不同，设备 0 结果正确，设备 1 结果错误

该用户环境下的工具使用流程如下：

1. 分别在设备 0 和设备 1 上安装 TorchDebug 工具的 .whl 包。
2. 分别在设备 0 和设备 1 上的模型中使用 TorchDebug 工具，插入相关接口代码，运行模型并 dump (保存) 得到网络训练过程的中间数据。
3. 调用 compare 接口用于对比保存下来的数据，计算误差并输出对比结果到 Excel 文件中。
4. 分析对比结果，找出导致误差大的算子。
5. 如果没有找到对应算子，则根据步骤 4 的对比结果确定导致误差大的大致范围，重新调整 dump 范围，再次 dump 数据并进行比较。

1.3.2 环境 2：相同模型，同一设备，不同软件栈版本的结果不同，旧版本结果正确，新版本结果错误

该用户环境下的工具使用流程如下：

1. 在该设备上安装 TorchDebug 工具的 .whl 包。
2. 在该设备上的模型中使用 TorchDebug 工具，插入相关接口代码。
3. 在当前版本 (新版本) 软件栈环境中运行模型，dump 得到网络训练过程的中间数据。
4. 卸载当前版本并安装旧版本软件栈，运行模型，保存得到网络训练过程的中间数据。
5. 调用 compare 接口用于对比保存下来的数据，计算误差并输出对比结果到 Excel 文件中。
6. 如果没有找到对应算子，则根据步骤 5 的对比结果确定导致误差大的大致范围，重新调整 dump 范围，再次 dump 数据并进行比较。

Note

步骤 3 和 4 对具体新旧软件栈版本保存数据的顺序没有特殊要求，只需确保旧版本的数据正确，新版本的数据错误即可。

1.4 工具使用注意事项

- 请确保两次 dump 过程调用的算子数量和算子顺序一致。
- 需要提前消除网络计算中的随机性和因算子算法差异导致的比对误差等干扰因素，如提前固定随机种子，dropout 参数需提前置 0 等。
- 不支持 dump torch.jit.script 和 torch.compile 后的 ScriptModule 和 ScriptFunction 的输入输出信息。
- 暂不支持如 Apex 等实现自定义 op 的输入输出信息。
- DP 模式下 dump 传递参数 rank 必须指定为 [0]，其它输入均视为非法输入。
- 对于显存占用较高的模型，使用工具 dump 数据时有可能出现 OOM 的现象，可能需要根据实际调整模型参数，如 batch_size 等。
- 目前的多进程启动方式要求在编写 compare() 脚本时，必须按照标准的 Python 脚本的写法 (无论是使用 CPU 还是 GPU)。

1.5 工具使用示例

1.5.1 插入代码示例说明

下面以在不同设备上对 ResNet50 进行 DDP 训练过程为例，对使用 TorchDebug 工具插入代码进行说明。

Shell 脚本：framework-tests/pt/resnet/run_resnet50_train_ddp.sh

Python 脚本：framework-tests/pt/resnet/main_ddp.py

在 Python 脚本中插入代码示例如下所示：

```
diff --git a/pt/resnet/main_ddp.py b/pt/resnet/main_ddp.py
index e6d8903f..ba38ff7b 100644
--- a/pt/resnet/main_ddp.py
+++ b/pt/resnet/main_ddp.py
@@ -18,6 +18,7 @@ import torch.utils.data
import torch.utils.data.distributed
import resnet as net
from keras_preprocessing import image
+from torchdebug import init, start, stop, set_seed

parser = argparse.ArgumentParser(description='PyTorch ImageNet Training')
@@ -169,6 +173,15 @@ def main_worker(gpu, ngpus_per_node, args):
    global best_acc1
    args.gpu = gpu

+    set_seed(1234, enable_dnn=True, benchmark=True, deterministic=True)

@@ -181,6 +182,11 @@ def main_worker(gpu, ngpus_per_node, args):
    args.rank = args.rank * ngpus_per_node + gpu
    dist.init_process_group(backend=args.dist_backend, init_method=args.dist_url,
                           world_size=args.world_size, rank=args.rank)

    # create model
    print("> creating model resnet50")
    print(
@@ -284,6 +290,9 @@ def main_worker(gpu, ngpus_per_node, args):
    top1_val = AverageMeter('Acc@1', ':6.2f')
    top5_val = AverageMeter('Acc@5', ':6.2f')

+    init(output_dir="./dump_dir0", level=2, skip=5, type="train", rank=[0]) // 在 epoch
+    ↪ 循环执行前进行初始化，设置 skip=5，每隔5次 dump 时机 dump 一次
    for epoch in range(args.start_epoch, args.epochs):
        if args.distributed:
            train_sampler.set_epoch(epoch)
@@ -332,6 +341,8 @@ def train(train_loader, model, criterion, optimizer, epoch, args,
    target = target.cuda(args.gpu, non_blocking=True)

    # compute output
+    if epoch == 0 and i == 0: // 限定 dump 的 epoch 和 step，如 epoch0 and step0，在
+    ↪ model 前向执行之前启动 start，激活 dump 功能
```

```

+         start()
        output = model(images)
        loss = criterion(output, target)

@@ -349,6 +360,7 @@ def train(train_loader, model, criterion, optimizer, epoch, args,
    # compute gradient and do SGD step
    optimizer.zero_grad()
    loss.backward()
+    stop()                                // 在调用 loss.backward() 之后调用 stop, 结束 dump
    optimizer.step()

    # measure elapsed time

```

上述的 dump 结果会保存到 ./dump_dir0 中。

得到在不同设备上 dump 的结果之后，将保存好的数据复制到 (scp 方式) 同一台主机上，编写脚本即可进行数据对比。示例如下所示：

假设正确结果的 dump 路径为 ./dump_dir0，有精度问题结果的 dump 路径为 ./dump_dir1，

```

from torchdebug import compare

compare("./dump_dir0", "./dump_dir1", "./output_dir")

```

对比结果将保存在 ./output_dir 中，每个 rank 的对比结果会保存为一个 JSON 文件和一个 Excel 文件，并以对应 rank 命名，如 rank_0.json，rank_0.xlsx。

对于插入代码的位置有一定的规律和要求，参考如下。

Important

- 需要注意的是，init, start, stop 3 个接口并不一定会添加到同一个文件中，需要根据实际的模型代码进行确定。
- 如果是多个文件，则需要每个文件上方都从 TorchDebug 导入相应的接口。某些模型中，start 接口的限定条件可能需要手动将代表 epoch/step 的变量传递过去。

插入代码 1:

```

from torchdebug import init, start, stop, set_seed

```

该代码的添加位置一般为 Python 脚本的最上方，与其它 import 包的位置相同。

插入代码 2:

```

set_seed(1234, enable_dnn=True, benchmark=True, deterministic=True)

```

该代码的添加位置一般为 Python 脚本中模型训练代码的最前面，比如上述 ResNet50 代码示例中是 main() -> main_worker()，所以添加在了 main_worker 最前面。

Note

该代码为非必要，在某些模型中可能已经固定了随机种子，可按需选择设置 `set_seed`。

插入代码 3:

```
init(output_dir="./dump_dir0", level=2, skip=5, type="train", rank=[0])
```

此为必要接口，参数可按需设置。该代码的添加位置一般为 Python 脚本中训练代码对 epoch 做 for 循环之前。

插入代码 4:

```
start()
```

此为必要接口，因为只需要 dump 一个 step 的数据，所以需要对 epoch 和 step 做出限定，比如上述 ResNet50 代码示例中是 `epoch == 0 and i == 0`，表示 dump 第 0 个 epoch 和第 0 个 step 的数据。该代码的添加位置一般为 Python 脚本中模型训练或者推理前向执行之前，比如上述 ResNet50 代码示例中是 `output = model(images)` 之前。

代码插入 5:

```
stop()
```

此为必要接口，不需要做限定条件。该代码的添加位置一般为 Python 脚本中模型训练反向结束或者推理结束之后且优化器更新之前，比如上述 ResNet50 代码示例中是 `loss.backward()` 之后。

插入代码 6:

```
compare("./dump_dir0", "./dump_dir1", "./output_dir")
```

不需要插入 Python 脚本，得到 2 组 dump 数据之后，新建一个 Python 文件，导入 compare 接口，直接使用即可。误差计算的基准值必须是正确数据，所以第一个参数必须是正确数据的 dump 路径，第二个参数必须是问题数据的 dump 路径。

1.5.2 手动模拟问题定位过程

这里，假设手动构造一个错误算子：以 `torch.nn.functional.max_pool2d` 算子为例，修改底层实现，将输出结果全部 +0.5。

为了方便测试，这里使用同一台机器并安装不同版本的 PyTorch (有无错误算子) 进行数据 dump 和对比，模拟问题的定位。测试步骤如下：

1. 在 `main_ddp.py` 中 `main_worker` 函数的开始处固定随机数 (main 函数中固定种子并没有生效，发现 `conv2d` 初始化生成的 weight 每次都不同)：

```
@@ -169,6 +170,13 @@ def main_worker(gpu, ngpus_per_node, args):
    global best_acc1
    args.gpu = gpu
```



```
+ # args.seed = 1234
+ # random.seed(args.seed)
+ # np.random.seed(args.seed)
+ # torch.manual_seed(args.seed)
+ # torch.cuda.manual_seed_all(args.seed)
+ # cudnn.deterministic = True
+ set_seed(1234, enable_dnn=True, benchmark=True, deterministic=True)

if args.gpu is not None:
    print("Use GPU: {} for training".format(args.gpu))
```

2. 根据上述插入代码的示例，保存不同版本 PyTorch 训练所 dump 的数据，无错误算子的数据保存至 ref_dir_skip5 中，有错误算子的数据保存至 debug_dir_skip5 中。

为了避免 dump 数据过多，可以适当调整 skip 参数的大小。这里设置 skip=5，表示每 5 个算子保存一次。再单独调用 compare 函数，对比结果保存至 ./out_skip5 中。

```
init(output_dir="./ref_dir_skip5", level=2, skip=5, type="train", rank=[0])
init(output_dir="./debug_dir_skip5", level=2, skip=5, type="train", rank=[0])
compare("./ref_dir_skip5", "./debug_dir_skip5", "./out_skip5")
```

在 ./out_skip5 中发现，首次出现误差较大的地方在 torch.nn.functional.conv2d.5.fwd.input,说明精度问题出现在 torch.nn.functional.conv2d.0.fwd 和 torch.nn.functional.conv2d.5.fwd 之间，可以缩小范围继续定位。

3. 根据步骤 2 中的结果，进一步确定出问题算子的大致范围，可以通过调整 op_range 和 op_list 来缩小 dump 的范围。

```
init(output_dir="./ref_dir_skip1", level=2, skip=1,
    ↪ op_range=["torch.nn.functional.conv2d.0.fwd", "torch.nn.functional.conv2d.
    ↪ 5.fwd"], type="train", rank=[0])

init(output_dir="./debug_dir_skip1", level=2, skip=1,
    ↪ op_range=["torch.nn.functional.conv2d.0.fwd", "torch.nn.functional.conv2d.
    ↪ 5.fwd"], type="train", rank=[0])

compare("./ref_dir_skip1", "./debug_dir_skip1", "./out_skip1")
```

缩小范围后的对比结果表明，torch.nn.functional.max_pool2d.4.fwd 的 input 一直，但是 output 误差较大。说明该算子的计算出现了问题，可以根据此 dtype 和 shape 进行复现，并进一步确定问题原因。

1.6 相关接口说明

1.6.1 set_seed

函数原型

```
set_seed(seed, enable_dnn, benchmark, deterministic) -> None
```

功能描述

固定所 dump 模型中的种子，去除随机性，可以选择式开启或关闭 dnn 后端相关的配置。

参数说明

- seed: 默认为 1234，表示固定随机数种子为 1234
- enable_dnn: 默认为 True，表示使用 cudnn 后端。设置为 False，将禁用 cudnn 后端，使用 torch native 实现
- benchmark: 默认为 False，表示使用启发式 (heuristic) 方法选择程序所认为的最合适的算法。设置为 True，表示使用穷尽式 (exhaustive search) 方法会遍历所有可选的算法进行比较而选出最优
- deterministic: 默认为 False，表示不强制使用 cudnn 确定性算法

建议使用方式

```
set_seed(1234, enable_dnn=True, benchmark=True, deterministic=True)
```

1.6.2 init

函数原型

```
init(model: nn.Module, output_dir: str = "./dump_dir", level: int = 2, dump_input: bool =
↳ True, dump_output: bool = True, dump_stack: bool = False, op_list: List[str] = [],
↳ op_range: List[str] = [], skip: int = 0, rank: List[int] = [], process_group = None,
↳ type: str = "train") -> None
```

功能描述

初始化 TorchDebug 工具。

参数说明

- model: 默认为 None，表示 dump 模型中所有算子。若配置该参数，model 应为 nn.Module 类型，表示只 dump nn.Module 的算子
- output_dir: str 类型，dump 数据的保存路径，默认存储在当前目录的 dump_dir 目录中
- level: int 类型，控制 dump 数据信息的详细程度，取值为 2, 1 和 0，分别对应 High, Middle 和 Low，默认为 High
- dump_input: bool 类型，是否 dump 算子的输入信息，默认为 True
- dump_output: bool 类型，是否 dump 算子的输出信息，默认为 True
- dump_stack: bool 类型，是否 dump 算子的 Python 调用栈信息，默认为 False。这里的调用栈是 hook 函数的调用栈，会与算子的输入或者输出信息一起 dump，保存到 txt 文件中
- op_list: List[str] 类型，控制 dump 的具体前向/反向算子，支持指定某一类 API，如 op_list=["relu"] 会包含 torch.relu, torch.relu_, torch.Tensor.relu, torch.nn.ReLU 和 torch.nn.functional.relu_ 等前向算子。默认为空列表，表示不限制算子
- op_range: List[str] 类型，控制 dump 信息的范围，只能传空列表，或者包含 Python API 信息的字符串/None 长度为 2 的列表，如 [], [None, "torch.nn.Dropout.175.fwd"], ["torch.nn.AdaptiveAvgPool2d.663.fwd", None]。控制范围前闭后开，默认为空列表，表示不限制 dump 范围

- None 表示不限制 dump 范围的开始或结束边界
- 包含 Python API 信息的字符串由 3 部分组成，包括算子的 API 名称，按照执行时间顺序对应的第几个前向算子，前向 fwd/反向 bwd
- skip: int 类型，表示每间隔 skip 次 dump 时机 (dump 时机一般指一个 torch 算子) dump 一次，默认为 1，表示会按顺序保存每一个 torch 算子的输入输出数据
- rank: List[int] 类型，默认为空列表，表示不限制
- process_group: torch.distributed.ProcessGroup 类型，表示需要 dump 的进程组。默认为 None，表示只会 dump 默认进程组。如果不是分布式场景，不需要设置该参数
- type: str 类型，用于区分是训练 (train) 或是推理 (inference)，默认为 train

1.6.3 start

函数原型

```
start() -> None
```

功能描述

开始 dump 数据。

参数说明

暂无。

1.6.4 stop

函数原型

```
stop() -> None
```

功能描述

停止 dump 数据。

参数说明

暂无。

1.6.5 compare

函数原型

```
compare(input_dir0: str, input_dir1: str, output_dir: str = "./", compare_bounds=[],
        use_multiprocess=False, num_processes=None, use_gpu=False, save_diff_details=False)
```

功能描述

按照如下误差计算公式，逐 Tensor 比较参照对象和 Debug 对象 2 个目录下 dump 的数据，并将结果输出到 Excel 文件中。

$$\begin{aligned} diff1 &= \frac{\sum |debug_data - base_data|}{\sum |base_data|} \\ diff2 &= \sqrt{\frac{\sum (debug_data - base_data)^2}{\sum base_data^2}} \\ cos_sim &= \frac{\sum base_data \cdot debug_data}{\sqrt{\sum base_data^2 \cdot \sum debug_data^2}} \end{aligned}$$

图 1: 误差计算公式

Excel 文件内容包含两者的 Name, Dtype, Shape, Mean, Var, Allclose, 最大绝对误差 MAE, 最大相对误差 MRE, Diff1, Diff2, 余弦相似度 Cos_sim, Nan 个数, Inf 个数, Max, Min 等。

参数说明

- input_dir0: str 类型，设备 0 (参照对象) 所 dump 数据的保存路径，无默认值
- input_dir1: str 类型，设备 1 (Debug 对象) 所 dump 数据的保存路径，无默认值
- output_dir: str 类型，比较结果的输出路径，默认输出到当前目录
- compare_bounds: list 类型，接受传递包含 int 类型元素的列表，元素数值为 dump 所得数据文件名称中的 ID 索引，如“torch.Tensor.mean.9.fwd.output.pt”中的“9”
 - 默认为空列表，表示对比所有的数据，支持传递 1 个数值，例如，[10]，表示值对比 ID 索引为 10 的数据文件；也支持传递 2 个数值，如 [10, 15]，表示对比 ID 索引从 10 至 15 的所有数据文件
 - 需要注意的是，该参数：
 - * 不支持传递负数的 int 值
 - * 不支持数值超出 dump 数据的索引范围
 - * 传递 2 个数值时，须确保非降序排列
- use_multiprocess: 表示是否使用多进程对比数据，默认为 False，即不使用
- num_processes: 在 use_multiprocess=True 的情况下，表示使用的进程数，即启动 num_processes 个进程进行数据对比。**注意：**在 use_multiprocess=False 的情况下，设置该参数无效
- use_gpu: 在 use_multiprocess=True 的情况下，表示使用 GPU 进行数据对比，默认为 False，即不使用 GPU。**注意：**

- 在 `use_multiprocess=False` 的情况下，设置该参数无效
- 在 `use_multiprocess=True` 和 `use_gpu=True` 的情况下，`num_processes` 表示使用的加速卡数量，默认 `num_processes=16`，您可以根据实际设备数量灵活设置
- `save_diff_details`: 获取 `diff_count` 统计数据的坐标信息，默认为 `False`。当设置为 `True` 时，表示将算子的名称和坐标信息保存成对应的 `pt` 文件，在 `output_dir` 指定的路径下根据 `input_dir` 中获取的 `rank` 数量创建对应的 `rank_x_diff_details` 目录，保存这些 `pt` 文件可供后续获取详细信息进行分析

更多说明

通过 Excel 文件中的误差评价指标进行分析，可以定位到出现精度误差的算子。

此时开发者可以发现，多个误差评价指标会被计算出，分析定位的过程需要对这些指标进行综合判断：最主要的判断指标为 `Allclose`, `MAE`, `Cos_sim`。一般出现问题的算子结果会直接体现在这 3 个指标中某个或者某几个的异常上。若 `Cos_sim` 值并不严格等于 1，而是一个十分接近 1 的数，同时 `Allclose=True`, `MAE=0.0`，此种情况认为算子数据也是正常的。若误差较小，可能 `Allclose=True`，但 `MAE != 0.0`，此时如果 `Cos_min` 等于 1 或者极为接近 1，数据一般是没有精度问题的；如果 `Cos_sim` 明显不等于 1，那此处的算子大概率是有问题的。

因此，精度问题的分析需要多个指标和对比数据的综合判断，一个指标的值并不能绝对反映算子是否存在精度问题。

如果某一行的计算结果显示由 `nan` 或者 `inf`，只需要关注 `Nan_count` 和 `Inf_count` 即可，检查该算子数据是否有 `nan/inf` 值。如果有，基本可以确定这个算子即为问题算子，然后尝试单独复现；如果没有，可能是数据计算越界导致的。此时对于其它指标可以不予关注。

Compare 对比结果以 Excel 文件形式展示，以下是对比项的说明：

- **Name**: 算子的名称，第几个前向算子，前向/反向，输入/输出，第几个参数
Name 对应保存的 `pt` 文件的 `name`，保存的 `pt` 文件类型为 `list`。以 `torch.nn.functional.conv2d` 为例，Python 端 `conv2d` 接口的参数一共有 7 个，分别为 (`x`, `weight`, `bias=None`, `stride=1`, `padding=0`, `dilation=1`, `groups=1`)，`bias=True` 时，保存的 `pt` 文件长度为 7，顺序分别对应这 7 个参数。Excel 文件中数据对比只包括 `Tensor` 类型数据，所以在表中会出现 `xxx.input.0`, `xxx.input.1`, `xxx.input.2` 的 `name`。
- **Dtype**: 数据类型
- **Shape**: 数据大小
- **Mean**: 均值
- **Var**: 方差
- **Allclose**: 在一定误差范围内 `RefObj` 和 `DebugObj` 是否接近
- **MAE**: 绝对误差最大值
- **MRE**: 绝对误差最大值处的相对误差
- **Round_count, Diff_count**: 对于 `FP32`, `FP16`, `BF16` 三种浮点数类型的按位比较的结果统计。`Round_count` 表示舍入误差的统计数量，`Diff_count` 表示精度误差的统计数量
- **Diff1, Diff2, Cos_min**: 误差评价公式计算结果
- **Nan_count**: `Tensor` 中 `nan` 的个数
- **Inf_count**: `Tensor` 中 `inf` 的个数
- **Max**: `Tensor` 的最大值
- **Min**: `Tensor` 的最小值

- Value: 保存的路径, 对应 pt 文件
- Stack: 算子的 Python 调用栈信息

1.6.6 analyze_op_diff_details

函数原型

```
analyze_op_diff_details(ref_dir, debug_dir, output_dir, ranks, op_list=None)
```

功能描述

根据保存有 diff_count 统计数据坐标信息的 pt 文件, 处理获取实际的基准值、错误值和误差, 并将其以字典的形式保存成输出路径下的 json 文件。单独提供处理数据接口的目的是为了 avoid compare 时统计的 diff_count 过大, 导致实时处理数据带来的时间开销。

参数说明

- ref_dir: str 类型, 设备 0 (参照对象) 所 dump 数据的保存路径, 无默认值
- debug_dir: str 类型, 设备 1 (Debug 对象) 所 dump 数据的保存路径, 无默认值
- output_dir: str 类型, 比较结果的输出路径, 默认输出到当前目录
- ranks: 表示处理某个或某些 rank 的数据, 类型为 list, 例如, rank=[0,1]
- op_list: 表示处理指定的算子, 类型为 list
 - 默认为 None, 处理 rank_x_diff_details 目录下所有 pt 对应的算子, 可以以列表格式传递待对比的算子, 例如, op_list=["torch.Tensor.mean.9.fwd.output.0", "torch.rsqrt.11.fwd.input.0", "torch.Tensor.type_as.13.fwd.input.0"]
 - 需要注意的是, 传递的算子格式和 Excel 表格中显示的 Name 需一致, 可以直接复制使用, 且只能传递表格中 diff_count 不为 0 的算子, 否则会找不到对应的 pt 文件

注意事项

该接口是专用于处理 diff_count 所统计的 diff_details, 如果进行 compare 时未传递 save_diff_details=True, 使用该接口时会报错。

1.7 框架算子支持

在天数智算软件栈 V4.3.0 中, TorchDebug 新增支持以下 dump flash_attn 框架算子 (包括大模型训练中通过 Megatron 或 ix-TransformerEngine 调用):

- flash_attn_func
- flash_attn_kvpacked_func
- flash_attn_qkvpacked_func
- flash_attn_varlen_func
- flash_attn_varlen_kvpacked_func
- flash_attn_varlen_qkvpacked_func
- flash_attn_with_kvcache

1.8 最佳实践

下面提供一个最佳实践，指导用户如何在进行大模型训练时使用 TorchDebug 进行精度调试。

使用方法可大致参考[插入代码示例说明](#)，但关键在于应在哪个位置设置 init, start, stop 操作开启数据的 dump；另外，在 dump 之前，建议先去除随机，可简化 case，减少 dump 的数据量，从而快速缩小出问题算子的范围。

那么，应该如何简化呢？您可以遵循以下操作步骤：

1. 排除随机 (此步骤很重要，因为有随机会影响 dump 到的数据是否判断正确)。当前没有随机算法的实现，都是确定性算法，不区分天数智芯的硬件平台，同一硬件运行两遍，所有带数值的评估指标都是 0.0。
2. 调整 DP1 PP1 TP1 在跑单加速卡时能否复现，此步骤相对也比较重要。因为多卡 dump 的数据量较大，建议先减少到单卡能否复现，这样才能减少 dump 的数据量，从而更快定位出错算子的范围。
3. 调整 global_batch_size=1 能否复现，此步骤尤其重要。因为如果一个 dp 内循环运行多个 microbatch，实际 dump 的数据时最后一个 microbatch 的数据，而期望应该 dump 第一个 microbatch 的数据才最为准确，至于如何指定只 dump 第一个 microbatch 的数据，目前还未找到好的方案。目前除了 VP，都支持设置 gbs=1，最新适配的 Megatron-LM 框架也支持 VP 同时设置 gbs=1。
4. 首次 dump 数据，建议合理设置 skip 参数，例如，设置 skip = 5，尽量减少 dump 的数据，这样可以快速对比，缩小出问题的算子范围。
5. 缩小出问题算子范围后，合理设置 op_list, op_range 参数，例如，op_list=["allreduce"], op_range=["torch.nn.functional.embedding.4.fwd.input.0", "torch.nn.functional.dropout.7.fwd.input.0"]。
6. 指定 iteration dump，无需 dump 所有 iteration，一般 dump 第一个 iteration 即可。
7. 在特定训练代码路径下添加 start 和 stop 操作，这个需要对 Megatron-LM 训练代码有深入的了解，才能更精确地设置 dump 数据地起始位置。此步骤建议不轻易使用，可以按照步骤 6 中的指导，dump 第一个 iteration 所有算子，或在第一个 iteration 中指定 op_list, op_range 减少待 dump 的算子。
8. 建议数据 dump 完成后，先检查 dump_dir 目录下的 pt 数量，并检查是否存在 transpose, permute, squeeze, view, reshape 等算子。这类算子一般不会出错，可以考虑不 dump。继而修改 /usr/local/lib/python3.10/dist-packages/torchdebug/wrap_ops.yaml，注释掉诸类算子即可。
9. 大部分通信算子都是直接对输入 tensor 进行修改，不创建新的输出 tensor。因此输入有可能存在问题，并非通信算子出现问题。所以，通过 Excel 表格发现通信算子有问题后，可以在模型代码中再进一步打印确认输入是否一致，而输出不一致。

下面是 init 操作，start 操作，stop 操作，以及观察 dump 后的数据的指导。

1. init 操作，可以在 megatron-lm-examples/pretrain_gpt.py:301 处添加。添加位置示例如下：

```
from torchdebug import *
...
if __name__ == "__main__":

    # Temporary for transition to core datasets
    train_valid_test_datasets_provider.is_distributed = True
    init(output_dir="./dump_dir0", level=2, skip=5, type="train", rank=[0])
    pretrain(
        train_valid_test_datasets_provider,
        model_provider,
        ModelType.encoder_or_decoder,
        forward_step,
```



```
args_defaults={'tokenizer_type': 'GPT2BPETokenizer'},
)
```

2. start 和 stop 操作，一般添加在 train_stop 前后，指定第一个 iteration 进行 dump。添加位置示例如下：

```
# /usr/local/lib/python3.10/site-packages/megatron/training/training.py
# $SW_HOME/local/lib64/python3/dist-packages/megatron/training/training.py

from torchdebug import *
...
if iteration == 1:
    start()
loss_dict, skipped_iter, should_checkpoint, should_exit, exit_code, grad_norm,
↪ num_zeros_in_grad = \
    train_step(forward_step_func,
               train_data_iterator,
               model,
               optimizer,
               opt_param_scheduler,
               config,
               do_eval)
if iteration == 1:
    stop()
```

3. 有了两次数据 (包括相同平台运行两遍以及不同平台各运行一遍) 后，进行 compare 操作，等待对比结束：

```
from torchdebug import *

compare("./dump_dir0", "./dump_dir1", "./output_dir")
```

对比结束后，将在 output_dir 目录下保存每个 rank 对应的 Excel 文件，导出至本地进行观察即可。示例中运行的是 DP4 PP1 TP1 的配置，因此会显示 4 张加速卡：

```
$ /home/iluvatar/tests/megatron-lm-examples/output_dir# ll
total 22252
drwxr-xr-x  2 root root    4096 Jun  6 22:31 ./
drwxr-xr-x 21 root root    4096 Jun  9 10:30 ../
-rw-r--r--  1 root root 5529353 Jun  6 21:52 rank_0.json
-rw-r--r--  1 root root 160593 Jun  6 21:52 rank_0.xlsx
-rw-r--r--  1 root root 5528873 Jun  6 21:32 rank_1.json
-rw-r--r--  1 root root 160516 Jun  6 21:32 rank_1.xlsx
-rw-r--r--  1 root root 5529035 Jun  6 22:31 rank_2.json
-rw-r--r--  1 root root 160451 Jun  6 22:31 rank_2.xlsx
-rw-r--r--  1 root root 5529787 Jun  6 22:11 rank_3.json
-rw-r--r--  1 root root 160644 Jun  6 22:11 rank_3.xlsx
```

4. 观察 dump 出来的 Excel 文件。

- a. 如果是随机性排除验证，同一硬件平台运行两遍进行对比，就观察对比后的表格数据中最后几列度量指标的数值是否都是 0.0。期望状态是没有任何误差，应保持一致才是合理的。关于 Allclose 指标，需要将 /usr/local/lib/python3.10/dist-packages/torchdebug/torchdebug/

config.py 中的 `ERROR_THRESHOLD` 和 `ERROR_THRESHOLD_FP16` 修改为 0.0。这样，只要有一丝误差，`Allclose` 就会显示 `False`。

例如，下面这张图中，所有前向和反向梯度结果的评估指标数值都是 0.0，而反向梯度更新时调用的 `all_reduce` 通信算子的结果并不是一模一样的，有较大误差。这就是不正常的，因为通信算子也没有随机算法；但值得注意的是，通信算子都是直接对输入 `tensor` 进行修改，不创建新的输出 `tensor`，因此输入有可能存在问题，并非通信算子出问题。

[illegible]

图 2: 示例数据

b. 排除了随机性后，即同一硬件平台分别运行两遍后结果一致，没有任何误差后，此时再对比不同硬件平台的比对结果。例如，竞品加速卡运行一遍，天玑 150 加速卡运行一遍，进行比对，或天玑 150 加速卡运行一遍，智铠 100 加速卡运行一遍，进行比对。这种情况下，就无法要求两边运行出来的结果一致，因为不同硬件平台存在细微误差。比对结束后，优先观察这两个评估指标即可。

- 1) 此时就不能以度量指标数值 0.0 来评估结果的正确性，一般由先查看 `cos_sim` 相似度。该指标相对更为准确判断出是否存在算子计算错误，通常 `cos_sim` 值都在 0.9999，1.0 才属于正常，只要存在错误，就会变成 0.88 甚至更低。
- 2) 通常相似度较为准确，基本能发现问题所在。除了相似度外，`Allclose` 指标也是一个常用的验证正确性的指标，但往往在实际模型训练中，累计绝对误差 `MAE` 会放到 0.1 左右，进而 `Allclose` 数值会变成 `False`，但也不一定是算错。

例如，下面这张图中，明显可以看出 Allclose 存在非常多 False，MAE 绝对误差最大已经达到了 0.04，此时会误导判断，但可以看到 cos_sim 基本都在 1.0。因此这种情况就是累计误差引起的，但不影响最终的收敛和 loss，grad_norm 均值。

[illegible]

图 3: 示例数据

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。